Inheritance – Example 1

```
class Account
{
    private int number;
    private String name;
    private double balance;

    public void setNumber(int number)
    {
        this.number = number;
    }

    public void setName(String name)
    {
        this.name = name;
    }

    public void setBalance(double balance)
    {
        this.balance = balance;
    }

    public int getNumber()
    {
        return number;
    }

    public String getName()
    {
        return name;
    }

    public double getBalance()
    {
        return balance;
    }

    public void deposit(double amount)
    {
        balance = balance + amount;
    }

    public void withdraw(double amount)
    {
        balance = balance - amount;
    }

    public void availableBalance()
```

```
    {
        System.out.println("Available Balance : " + balance);
    }

    public void display()
    {
        System.out.println("Account Number : " + number);
        System.out.println("Customer Name  : " + name);
    }
}



class CurrentAccount extends Account
{

}



class SavingsAccount extends Account
{

}



public class AccountMain
{
    public static void main(String[] args)
    {
        // Savings Account
        SavingsAccount savings = new SavingsAccount();

        savings.setNumber(1001);
        savings.setName("David");
        savings.setBalance(10000);

        savings.deposit(2000);
        savings.withdraw(1000);

        System.out.println("----- Savings Account -----");
        savings.display();
        savings.availableBalance();



        // Current Account
```

```
        CurrentAccount current = new CurrentAccount();

        current.setNumber(1002);
        current.setName("Diya");
        current.setBalance(20000);

        current.deposit(5000);
        current.withdraw(3000);

        System.out.println("\n----- Current Account -----");
        current.display();
        current.availableBalance();


    }
}
```

Constructor types

```
class Employee
{
    private int id;
    private String name;
    private int age;


    public Employee()
    {
       System.out.println("Inside default constructor");
    }

    // Parameterized Constructor

    public Employee(int id, String name, int age) {
        System.out.println("Inside p.c");
        this.id = id;
        this.name = name;
        this.age = age;
    }


    // Getter and Setter for id
    public int getId() {
        return id;
    }
```

```java
    public void setId(int id) {
        this.id = id;
    }

    // Getter and Setter for name
    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    // Getter and Setter for age
    public int getAge() {
        return age;
    }

    public void setAge(int age) {
        this.age = age;
    }

    // Method to display employee details
    public void displayDetails() {
        System.out.println("ID: " + getId());
        System.out.println("Name: " + getName());
        System.out.println("Age: " + getAge());
    }
}



public class EmployeeDemo {
    public static void main(String[] args) {
        // Creating Employee object using Parameterized Constructor


       Employee employee = new Employee();
       employee.setId(10);
       employee.setName("John");
       employee.setAge(28);;



       Employee emp = new Employee(101, "Anu", 28);
       emp.displayDetails();
```

```
        emp.setName("Bob");
        emp.setAge(30);

       // Displaying updated details
       // System.out.println("Updated Employee Details:");
       // emp.displayDetails();
    }
}
```

Constructor Rules

```
//Example 1
class Student
{
    int id;
    String name;
    int age;

    void display()
    {
        System.out.println("Employee ID   : " + id);
        System.out.println("Employee Name : " + name);
        System.out.println("Employee Age  : " + age);
    }
}

public class StudentMain
{
        public static void main(String[] args)
        {
            Student emp = new Student();

            emp.id = 101;
            emp.name = "David";
            emp.age = 25;

            emp.display();
         }
}
```

```
//Example 2
class Employee
{
    int id;
    String name;
```

```
    int age;

    public Employee()
    {

    }
    // Parameterized Constructor
    Employee(int id, String name, int age)
    {
        this.id = id;
        this.name = name;
        this.age = age;
    }

    void display()
    {
        System.out.println("Employee ID   : " + id);
        System.out.println("Employee Name : " + name);
        System.out.println("Employee Age  : " + age);
    }
}

public class EmployeeMain
{
    public static void main(String[] args)
    {
        Employee emp1 = new Employee(101, "David", 25);
        Employee emp2 = new Employee(102, "Rahul", 30);

        emp1.display();

        System.out.println();

        emp2.display();

        Employee employee = new Employee();
    }
}
```

Method overloading

```
class MethodOverload1
{

	public void add(int a,int b)
	{
		int Sum1 = a + b;
		System.out.println("\n The method add(int a,int b) is
being called: ");
		System.out.println("\n The Sum of the two Numbers : " +
Sum1);
	}

	public void add(int a,int b,int c)
	{
		int Sum2 = a + b + c;
		System.out.println("\n The method add(int a,int b,int
c) is being called: ");
		System.out.println("\n The Sum of the three Numbers  "
+ Sum2);
	}
}

public class MethodOverloadDemo1
{
 public static void main(String args[])
 {
		MethodOverload1 sum= new MethodOverload1();
		sum.add(2,2);
		sum.add(2,2,2);
 }
}
```

```
class MethodOverload2
{
     //public int a=5;
     //public char b='A';

     public void display(int a,char b)
     {
       System.out.println("\n The method add(int a,char b)
is being called: ");
```

```
       System.out.println("\n The Value of int a & char b
are " + a+","+b);
     }


     public void display(char b,int a)
     {

       System.out.println("\n The method add(char b,int a)
is being called: ");
       System.out.println("\n The Value of char b & int a
are " + b+"," + a);
     }

}

public class MethodOverloadDemo2
{
 public static void main(String args[])
 {
     MethodOverload2  obj = new MethodOverload2();
     obj.display('A', 25);
     obj.display(2,'B');
 }
}




class MethodOverload3
{
      public void add(int a,int b)
      {
          int Sum1 = a + b;
          System.out.println("\n The method add(int a,int
b) is being called: ");
          System.out.println("\n The Sum of the two Numbers
is " + Sum1);
      }

      public void add(int a,double b)
```

```
      {
          double Sum2 = a + b ;
          System.out.println("\n The method add(int
a,double b) is being called: ");
          System.out.println("\n The Sum of the three
Numbers is " + Sum2);
      }

}

public class MethodOverloadDemo3
{
      public static void main(String args[])
      {
            MethodOverload3 sum= new MethodOverload3();
            sum.add(2,2);
            sum.add(3,0.1);
       }
}



 //method overloading with diff  return type is allowed.
class MethodOverload4
{
     public int add(int a,int b)
      {
          int sum1 = a + b;
          System.out.println("\nThe method int add(int
a,int b) is being called:");
          System.out.println("\n The Sum of the two Numbers
" + sum1);
          return(sum1);
      }

      public float add(int a,int b,float c)
      {
          float sum2 = a + b + c;
          System.out.println("\n The method double add(int
a,int b) is being called:");
```

```
            System.out.println("\n The Sum of the two Numbers
" + sum2);
            return(sum2);
        }
}

public class MethodOverloadDemo4
{
 public static void main(String args[])
 {
     MethodOverload4 sum = new MethodOverload4();
     sum.add(3,3);
     sum.add(2,2,3.0f);
 }
}
```